Listat dhe Tuplet në Python

Leksion — Përmbledhje e plotë sipas librit

Alion Emini

kurse.informatika
Kurs i Programimit në Python

September 30, 2025

Përmbajtja

1. Sekuenca dhe Hyrje në Lista
2. Operatori * dhe Iterimi
3. Indeksimi dhe `len()`
4. Mutability, Bashkimi dhe Slicing
5. Kërkimi, Kopjimi dhe Përpunimi
6. Punë me lista dhe skedarë
7. List Comprehensions
8. Metodat e Listave & Funksione të dobishme
9. Listat Dy-Dimensionale
10. Tuplet

Objektivat e kapitullit

Do të mësosh të:

- Kuptosh *sekuencat* dhe dallimin **listë** (mutable) vs **tuple** (immutable).
- Krijosh/shfaqësh lista & tupla; `len()` , indeksim (edhe negativ), iterim `for`/`while`.
- Përdorësh **slicing** `L[start:end:step]` (& rregullat e kufijve, indekse negative).
- Aplikosh operatorët `+` , `+=` , `*` , `in` / `not in` .
- **Kopjosh** lista (5 mënyra) dhe të kuptosh *shallow copy*.
- Përpunosh lista: shuma, mesatarja, `min` / `max` ; kalimi/kthimi i listave nga funksione.
- Përdorësh **list comprehensions** (transformim, filtër, kusht).
- Punosh me **skedarë**: `writelines` , `write` / `print` , `readlines` + `rstrip` , `int` / `float` .
- Ndërtosh **lista 2D** dhe të shmangësh kurthin `[[0]*C]*R` .
- Njohësh gabimet e shpeshta: `IndexError` , `ValueError` , `ZeroDivisionError` .

Përmbajtja

1. Sekuenca dhe Hyrje në Lista
2. Operatori * dhe Iterimi
3. Indeksimi dhe `len()`
4. Mutability, Bashkimi dhe Slicing
5. Kërkimi, Kopjimi dhe Përpunimi
6. Punë me lista dhe skedarë
7. List Comprehensions
8. Metodat e Listave & Funksione të dobishme
9. Listat Dy-Dimensionale
10. Tuplet

Sekuenca (Sequences)

Përkufizim

Sekuencë = objekt që mban elemente të renditura dhe lejon lexime/manipulime.

- **Listat** — *mutable*, elementët mund të ndryshohen.
- **Tuplet** — *immutable*, s'ndryshohen pas krijimit.

Krijimi i listave

Sintaksa

Listat krijohen me kllapa katrore `[...]` dhe mund të përmbajnë tipe të ndryshme.

```
even_numbers = [2, 4, 6, 8, 10]
```

2	4	6	8	10

Shfaqja e listës

Shënim

Funksioni `print()` shfaq listat me kllapa katrore dhe elementët të ndarë me presje.

```
even_numbers = [2, 4, 6, 8, 10]
print(even_numbers)
```

Output:

```
[2, 4, 6, 8, 10]
```

Listë me string

Shembull

Listat mund të përmbajnë edhe tekst (`str`).

```
names = ['Molly', 'Steven', 'Will', 'Alicia', 'Adriana']
print(names)
```

Output:

```
['Molly', 'Steven', 'Will', 'Alicia', 'Adriana']
```

Molly	Steven	Will	Alicia	Adriana

Listë me elementë të përzier

Shembull

Një listë mund të përmbajë tipe të ndryshme të dhënash.

```
info = ['Alicia', 27, 1550.87]
print(info)
```

Output:

```
['Alicia', 27, 1550.87]
```

Alicia	27	1550.87

Funksioni `list()`

Përkufizim

`list()` është një funksion ndërtues që kthen një listë të re nga çdo *iterable* (p.sh. string, tuple, range).

```
print(list("hello"))
```

Output:

```
['h', 'e', 'l', 'l', 'o']
```

Shënim

`list()` përdoret për të konvertuar çdo iterable në listë ose për të bërë një kopje të listës ekzistuese.

range(stop) — vetëm fundi

Teori

range(stop) krijon një sekuencë nga 0 deri stop-1.

```
numbers = list(range(5))
```

0	1	2	3	4

`range(start, stop)` — fillimi dhe fundi

Teori

`range(start, stop)` krijon një sekuencë nga `start` deri `stop-1`.

```
numbers = list(range(2, 7))
```

2	3	4	5	6

`range(start, stop, step)` — me hap

Teori

`range(start, stop, step)` krijon një sekuencë nga `start` deri `stop-1`, duke shtuar `step` çdo herë.

```
numbers = list(range(1, 10, 2))
```

1	3	5	7	9

Përmbajtja

1. Sekuenca dhe Hyrje në Lista
2. Operatori * dhe Iterimi
3. Indeksimi dhe `len()`
4. Mutability, Bashkimi dhe Slicing
5. Kërkimi, Kopjimi dhe Përpunimi
6. Punë me lista dhe skedarë
7. List Comprehensions
8. Metodat e Listave & Funksione të dobishme
9. Listat Dy-Dimensionale
10. Tuplet

Operatori i përsëritjes *

Teori

Operatori * përdoret për të përsëritur një listë disa herë dhe për të krijuar një listë të re me elementët e përsëritur.

- `L * n` — kthen një listë të re me elementët e `L` të përsëritur `n` herë.
- `n` duhet të jetë numër i plotë jo-negativ.

```
lista_e_re = lista_ekzistuese * n
```

Shembull

`[0] * 4` $\longrightarrow$ `[0, 0, 0, 0]`

Shembull i përsëritjes *

```
numbers = [0] * 5
numbers2 = [1, 2, 3] * 3
```

0	0	0	0	0

1	2	3	1	2	3	1	2	3

Shënim

`list * n` krijon *n* kopje të listës dhe i bashkon.

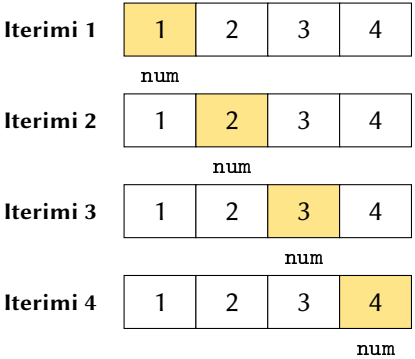
Iterimi me `for`

```python
numbers = [1, 2, 3, 4]
for num in numbers:
    print(num)
```

Output:

```
1
2
3
4
```

Iterimi me `for`



Shembull: nuk ndryshon listën

Rëndësishme

Variabla e ciklit mban *kopje* të vlerës — ricaktimi `num = 99` nuk ndryshon listën.

```
numbers = [1, 2, 3, 4]
for num in numbers:
    num = 99
print(numbers)
```

Output:

```
[1, 2, 3, 4]
```

Përmbajtja

1. Sekuenca dhe Hyrje në Lista
2. Operatori * dhe Iterimi
3. Indeksimi dhe `len()`
4. Mutability, Bashkimi dhe Slicing
5. Kërkimi, Kopjimi dhe Përpunimi
6. Punë me lista dhe skedarë
7. List Comprehensions
8. Metodat e Listave & Funksione të dobishme
9. Listat Dy-Dimensionale
10. Tuplet

Indeksimi në listë

Teori

Çdo element në listë ka një indeks: fillon nga 0 (pozitiv) ose -1 (negativ, nga fundi).

```
my_list = [10, 20, 30, 40]
```

0	1	2	3
10	20	30	40
-4	-3	-2	-1

Modifikimi i elementeve përmes indeksit

```
my_list = [10, 20, 30, 40]
my_list[2] = 99
```

0	1	2	3
10	20	99	40

Iterim me indeks pozitiv

Teori

Indekset pozitive fillojnë nga 0 dhe rriten deri në fund të listës.

```
my_list = [10, 20, 30, 40]
index = 0
while index < 4:
    print(my_list[index])
    index += 1
```

0	1	2	3
10	20	30	40

Indekse negative

Teori

Indekset negative fillojnë nga -1 (elementi i fundit) dhe shkojnë drejt majtas.

```
my_list = [10, 20, 30, 40]
print(my_list[-1], my_list[-2], my_list[-3], my_list[-4])
```

Output:

```
40 30 20 10
```

10	20	30	40
-4	-3	-2	-1

Gabimi IndexError

Teori

IndexError ndodh kur përpiqesh të aksesosh një indeks që nuk ekziston në listë.

```
my_list = [10, 20, 30, 40]

index = 0
while index < 5:
    print(my_list[index]) # -> IndexError kur index == 4
    index += 1
```

Kujdes

Indekset vlejnë nga 0 deri në len(list)-1.

Output-i i IndexError

```
10
20
30
40
Traceback (most recent call last):
  File "script.py", line 5, in <module>
    print(my_list[index])
IndexError: list index out of range
```

Përdorimi i len() për siguri

Teori

Funksioni len() kthen numrin e elementeve në listë. Përdore për të kontrolluar kufijtë dhe shmangur gabime.

```
index = 0
while index < len(my_list):
    print(my_list[index])
    index += 1
```

Këshillë

Gjithmonë përdor len(list) në cikle për të shmangur IndexError.

Iterim sipas indeksit me range(len(...))

```python
names = ['Jenny', 'Kelly', 'Chloe', 'Aubrey']
for index in range(len(names)):
    print(names[index])
```

Output:

```
Jenny
Kelly
Chloe
Aubrey
```

Përmbajtja

1. Sekuenca dhe Hyrje në Lista
2. Operatori * dhe Iterimi
3. Indeksimi dhe `len()`
4. Mutability, Bashkimi dhe Slicing
5. Kërkimi, Kopjimi dhe Përpunimi
6. Punë me lista dhe skedarë
7. List Comprehensions
8. Metodat e Listave & Funksione të dobishme
9. Listat Dy-Dimensionale
10. Tuplet

Listat janë *mutable*

```python
print(numbers)
numbers[0] = 99
```

Teori

Listat janë *mutable* — elementët mund të ndryshohen pas krijimit.

Para	1	2	3	4	5
Pas	99	2	3	4	5

Mbushje me indekse

```python
numbers = [0] * 5
for i in range(len(numbers)):
    numbers[i] = 99
print(numbers)
```

Output:

```
[99, 99, 99, 99, 99]
```

Programi 1 — sales_list.py

```python
NUM_DAYS = 5

def main():
    sales = [0] * NUM_DAYS
    print('Enter the sales for each day.')
    for index in range(len(sales)):
        sales[index] = float(input(f'Day #{index + 1}: '))
    print('Here are the values you entered:')
    for value in sales:
        print(value)
```

Output i Programit 1 - sales_list.py

```
Enter the sales for each day.
Day #1: 150.75
Day #2: 200.50
Day #3: 175.25
Day #4: 300.00
Day #5: 225.80
Here are the values you entered:
150.75
200.5
175.25
300.0
225.8
```

Bashkimi i listave me +

```
list1 = [1, 2, 3, 4]
list2 = [5, 6, 7, 8]
list3 = list1 + list2 # listë e re
```

1	2	3	4	5	6	7	8

Bashkimi me += (ndryshon listën ekzistuese)

```
list1 = [1, 2, 3, 4]
list2 = [5, 6, 7, 8]
list1 += list2
```

1	2	3	4	5	6	7	8

Kujdes

Konkatenimi funksionon *vetëm* listë me listë.

Slicing: marrja e një pjese të listës

Teori

Slicing lejon të marrësh një nën-listë nga një listë ekzistuese duke përdorur sintaksën `L[start:end]`. Kjo kthen elementët nga indeksi `start` deri në `end-1`.

```
days = ['Sunday','Monday','Tuesday','Wednesday','Thursday','Friday','Saturday']
print(days[2:5])
```

Sunday	Monday	Tuesday	Wednesday	Thursday	Friday	Saturday

Slicing: fillimi dhe fundi

Teori

Slicing me `L[start:end]` kthen elementët nga indeksi `start` deri në `end-1`. Këtu, `numbers[1:3]` merr elementët me indekse 1 dhe 2.

```
numbers = [1,2,3,4,5]
print(numbers[1:3])
```

1	2	3	4	5

Slicing: pa fillim (`L[:end]`)

Teori

Nëse lë mungesë `start`, `L[:end]` merr nga fillimi deri në `end-1`.

```
numbers = [1,2,3,4,5]
print(numbers[:3])
```

1	2	3	4	5

Slicing: pa fund (`L[start:]`)

Teori

Nëse lë mungesë end, `L[start:]` merr nga `start` deri në fund.

```
numbers = [1,2,3,4,5]
print(numbers[2:])
```

1	2	3	4	5

Slicing: kopje e plotë (`L[:]`)

Teori

`L[:]` merr të gjithë listën (kopje).

```
numbers = [1,2,3,4,5]
print(numbers[:])
```

1	2	3	4	5

Slicing me `step`

Teori

Sintaksa `L[start:end:step]` kthen çdo `step`-t element nga `start` deri në `end-1`.
Mund të përdorësh edhe indekse negative dhe hap negativ për të marrë listën mbrapsht.

```
numbers = [1,2,3,4,5,6,7,8,9,10]
print(numbers[1:8:2])
```

1	2	3	4	5	6	7	8	9	10

Slicing me indekse negative: end përtej fundit

Teori

Nëse end është më i madh se gjatësia e listës, Python ndalon te fundi i listës pa gabim.

```
numbers = [1,2,3,4,5,6,7,8,9,10]
print(numbers[-5:20])
```

1	2	3	4	5	6	7	8	9	10

Rregull

end përtej fundit $\Rightarrow$ përdoret gjatësia e listës.

Slicing me indekse negative: `start` para fillimit

Teori

Nëse `start` është më i vogël se 0, Python fillon nga fillimi i listës.

```
numbers = [1,2,3,4,5,6,7,8,9,10]
print(numbers[-20:3])
```

1	2	3	4	5	6	7	8	9	10

Rregull

`start` para fillimit $\Rightarrow$ përdoret 0.

Slicing me indekse negative: start > end

Teori

Nëse start është më i madh se end dhe step është pozitiv, rezultati është listë bosh.

```
numbers = [1,2,3,4,5,6,7,8,9,10]
print(numbers[7:3])
```

Rregull

start > end ⇒ listë bosh.

Përmbajtja

1. Sekuenca dhe Hyrje në Lista
2. Operatori * dhe Iterimi
3. Indeksimi dhe `len()`
4. Mutability, Bashkimi dhe Slicing
5. Kërkimi, Kopjimi dhe Përpunimi
6. Punë me lista dhe skedarë
7. List Comprehensions
8. Metodat e Listave & Funksione të dobishme
9. Listat Dy-Dimensionale
10. Tuplet

Operatori in në lista (Programi 2 - in_list.py)

Teori

in kontrollon nëse një element ekziston në listë.

```
def main():
    prod_nums = ['V475', 'F987', 'Q143', 'R688']
    search = input('Enter a product number: ')
    if search in prod_nums:
        print(f'{search} was found in the list.')
    else:
        print(f'{search} was not found in the list.')
```

Output:

```
Enter a product number: Q143
Q143 was found in the list.
```

Operatori `not in` në lista

Teori

`not in` kontrollon nëse një element *nuk* ekziston në listë.

```
prod_nums = ['V475', 'F987', 'Q143', 'R688']
if 'X999' not in prod_nums:
    print('X999 nuk gjendet në listë.')
```

Output:

```
X999 nuk gjendet në listë.
```

Pse `list1 = list2` **nuk** është kopjim? I

Shpjegim

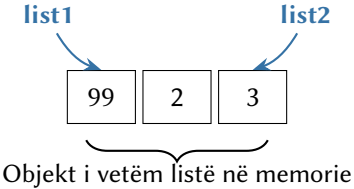
Në Python, shprehja `list1 = list2` **nuk** krijon një kopje të re të listës. Të dyja variablat referojnë të *njëjtën* listë në memorje. Çdo ndryshim përmes njërës reflektohet edhe te tjetra.

```
list2 = [1, 2, 3]
list1 = list2
list1[0] = 99
print(list1)
print(list2)
```

Output:

```
[99, 2, 3]
[99, 2, 3]
```

Pse `list1 = list2` **nuk** është kopjim? - Illustrim I



Kopjimi i listave me `.copy()`

Teori

Metoda `.copy()` kthen një kopje të re të listës (shallow copy).

```
list1 = [1, 2, 3]
list_a = list1.copy()
```

Shënim

Kopjohen vetëm elementët e nivelit të parë; nën-listat janë të përbashkëta.

Kopjimi i listave me `list()`

Teori

Funksioni ndërtues `list()` kthen një listë të re nga një iterable ekzistuese.

```
list1 = [1, 2, 3]
list_b = list(list1)
```

Kopjimi i listave me slicing [:]

Teori

`list1[:]` kthen një kopje të plotë të listës (nga fillimi në fund).

```
list1 = [1, 2, 3]
list_c = list1[:]
```

Kopjimi i listave me `for/append`

Teori

Mund të kopjosh elementët një nga një me cikël dhe append.

```
list1 = [1, 2, 3]
list_d = []
for i in list1:
    list_d.append(i)
```

Kopjimi i listave me + []

Teori

Bashkimi i një liste bosh me një listë ekzistuese kthen një kopje të re.

```
list1 = [1, 2, 3]
list_e = [] + list1
```

Programi 3 (total_list.py)

```python
def main():
    numbers = [2, 4, 6, 8, 10]

    # Variabël akumuluese.
    total = 0

    # Mblidh vlerat e listës.
    for value in numbers:
        total += value

    print(f'The total of the elements is {total}.')
```

```
The total of the elements is 30.
```

Programi 4 (average_list.py)

```python
def main():
    scores = [2.5, 7.3, 6.5, 4.0, 5.2]

    total = 0.0
    for value in scores:
        total += value

    average = total / len(scores)
    print(f'The average of the elements is {average}.')
```

```
The average of the elements is 5.1.
```

Programi 5 (total_function.py)

```python
def main():
    numbers = [2, 4, 6, 8, 10]
    print(f'The total is {get_total(numbers)}.')

# get_total pranon një listë dhe kthen shumën e elementëve.
def get_total(value_list):
    total = 0
    for num in value_list:
        total += num
    return total
```

```
The total is 30.
```

Programi 6 (return_list.py)

```python
def main():
    numbers = get_values()

    print('The numbers in the list are:')
    print(numbers)

def get_values():
    values = []
    again = 'y'

    while again.lower() == 'y':
        num = int(input('Enter a number: '))
        values.append(num)

        print('Do you want to add another number?')
        again = input('y = yes, anything else = no: ')
        print()
```

Output i Programit 6

```
Enter a number: 5
Do you want to add another number?
y = yes, anything else = no: y

Enter a number: 8
Do you want to add another number?
y = yes, anything else = no: n

The numbers in the list are:
[5, 8]
```

Zgjedhje rastësore me `random.choice`

Teori

`random.choice(lista)` kthen një element të rastësishëm nga lista.

```
import random
names = ['Jenny', 'Kelly', 'Chloe', 'Aubrey']
winner = random.choice(names)
```

```
Aubrey
```

```
Kelly
```

Zgjedhje me përsëritje: `random.choices`

Teori

`random.choices(lista, k=n)` kthen një listë me `n` elementë të zgjedhur rastësisht (me përsëritje).

```
import random
nums = [1,2,3,4,5,6,7,8,9,10]
selected = random.choices(nums, k=3)
```

```
[2, 5, 2]
```

```
[7, 1, 7]
```

Zgjedhje pa përsëritje: `random.sample`

Teori

`random.sample(lista, k=n)` kthen një listë me `n` elementë unikë të zgjedhur rastësisht (pa përsëritje).

```
import random
nums = [1,2,3,4,5,6,7,8,9,10]
unique = random.sample(nums, k=3) # elementë unikë
```

```
[3, 7, 1]
```

```
[10, 2, 5]
```

Përmbajtja

1. Sekuenca dhe Hyrje në Lista
2. Operatori * dhe Iterimi
3. Indeksimi dhe `len()`
4. Mutability, Bashkimi dhe Slicing
5. Kërkimi, Kopjimi dhe Përpunimi
6. Punë me lista dhe skedarë
7. List Comprehensions
8. Metodat e Listave & Funksione të dobishme
9. Listat Dy-Dimensionale
10. Tuplet

Përse punojmë me lista dhe skedarë?

Shpjegim

Shpesh, të dhënat që ruhen në listë duhen ruajtur në disk për përdorim të mëvonshëm, ose duhen lexuar nga një skedar për t'u përpunuar në program. Kjo është veçanërisht e dobishme për ruajtjen e të dhënave të përdoruesit, konfigurimeve, rezultateve të llogaritjeve, etj.

Programi 7 (`writelines.py`) — Shkrimi i një liste me `writelines` I

Përkufizim

`writelines(lista)` është një metodë e objektit skedar që shkruan të gjithë elementët e listës në skedar, njëri pas tjetrit, **pa** shtuar automatikisht ndarës rreshti (`\n`).

```
def main():
    cities = ['New York', 'Boston', 'Atlanta', 'Dallas']
    with open('cities.txt', 'w', encoding='utf-8') as outfile:
        outfile.writelines(cities) # -> të gjitha në një rresht!
```

Përmbajtja e skedarit `cities.txt`:

```
New YorkBostonAtlantaDallas
```

Programi 7 (`writelines.py`) — Shkrimi i një liste me `writelines` II

Kujdes

`writelines` **nuk** shton ' `\n` ' vetë, prandaj elementët do të bashkohen në një rresht.

Programi 8 (`write_list.py`) — Një element për rresht me `write`

Përkufizim

Për të shkruar çdo element të listës në një rresht të veçantë në skedar, duhet të shtosh manualisht karakterin e ri-rreshtit (`\n`) pas çdo elementi.

```
def main():
    cities = ['New York', 'Boston', 'Atlanta', 'Dallas']
    with open('cities.txt', 'w', encoding='utf-8') as outfile:
        for item in cities:
            outfile.write(item + '\n')
```

Output i Programit 8 (`write_list.py`)

Përmbajtja e skedarit `cities.txt`**:**

```
New York
Boston
Atlanta
Dallas
```

Programi 9 (`write_list_print.py`) — Një element për rresht me `print`

Shënim

Nëse përdor `print(item, file=outfile)`, `print` shton automatikisht '`\n`' në fund të çdo thirrjeje.

```
def main():
    cities = ['New York', 'Boston', 'Atlanta', 'Dallas']
    with open('cities.txt', 'w', encoding='utf-8') as outfile:
        for item in cities:
            print(item, file=outfile)
```

Output i Programit 9 (`write_list_print.py`)

Përmbajtja e skedarit `cities.txt`**:**

```
New York
Boston
Atlanta
Dallas
```

Pse përdorim `print` me `file=outfile`?

Funksioni `print` është më i thjeshtë për të shtuar automatikisht ndarësin e rreshtit dhe është më i sigurt për të shmangur gabime të zakonshme.

Programi 10 (`read_list.py`) — Leximi në listë me `readlines` dhe `rstrip`

Përkufizim

`readlines()` lexon të gjithë rreshtat nga një skedar tekst dhe i kthen si listë *string*-ësh, ku secili element përmban një rresht (përfshirë karakterin ' `\n` ' në fund).

```
def main():
    with open('cities.txt', 'r', encoding='utf-8') as infile:
        cities = infile.readlines()

    # Hiq '\n' nga çdo element
    for i in range(len(cities)):
        cities[i] = cities[i].rstrip('\n')

    print(cities)
```

Output i Programit 10 (read_list.py)

Output:

```
['New York', 'Boston', 'Atlanta', 'Dallas']
```

Pse përdorim rstrip?

Kur lexon rreshta nga skedari, secili element i listës përfundon me ' \n '. rstrip('\n') heq këtë karakter që të marrësh vetëm tekstin e pastër.

Programi 11 (`write_number_list.py`) — Shkrimi i numrave në skedar

Përkufizim

Kur shkruan numra në skedar tekst, duhet t'i konvertosh në *string* (me `str()` ose `f-string`) dhe të shtosh '`\n`' për çdo rresht.

```
def main():
    numbers = [1, 2, 3, 4, 5, 6, 7]
    with open('numberlist.txt', 'w', encoding='utf-8') as outfile:
        for item in numbers:
            outfile.write(f'{item}\n') # ose str(item) + '\n'
```

Output i Programit 11 (`write_number_list.py`)

Përmbajtja e skedarit `numberlist.txt`:

```
1
2
3
4
5
6
7
```

Programi 12 (`read_number_list.py`) — Leximi dhe konvertimi i numrave

Përkufizim

Kur lexon numra nga skedari, ata vijnë si *string*-je. Duhet t'i konvertosh në `int` ose `float` për t'i përdorur si numra në program.

```
def main():
    with open('numberlist.txt', 'r', encoding='utf-8') as infile:
        numbers = infile.readlines()

    for i in range(len(numbers)):
        numbers[i] = int(numbers[i]) # heq '\n' automatikisht gjatë int()

    print(numbers)
```

Output i Programit 12 (read_number_list.py)

Output:

```
[1, 2, 3, 4, 5, 6, 7]
```

Përmbledhje: Lista dhe Skedarë

- Për të ruajtur një listë në skedar, përdor `write`, `writelines`, ose `print(..., file=...)`.
- Për të lexuar një listë nga skedari, përdor `readlines()` dhe pastro elementët me `rstrip()` ose konvertoji në tipin e duhur.
- Kujdes me ndarësit e rreshtave dhe konvertimin e tipeve!

Përmbajtja

1. Sekuenca dhe Hyrje në Lista
2. Operatori * dhe Iterimi
3. Indeksimi dhe `len()`
4. Mutability, Bashkimi dhe Slicing
5. Kërkimi, Kopjimi dhe Përpunimi
6. Punë me lista dhe skedarë
7. List Comprehensions
8. Metodat e Listave & Funksione të dobishme
9. Listat Dy-Dimensionale
10. Tuplet

List Comprehension — Teoria

Çfarë është?

List comprehension është një mënyrë e shkurtër dhe e fuqishme për të krijuar lista të reja duke përpunuar elementët e një iterable (listë, range, etj.), me ose pa filtër.

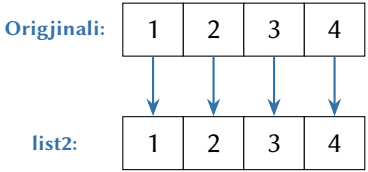
- Sintaksa bazë: `[expression for item in iterable]`
- Mund të shtosh kusht: `[expression for item in iterable if condition]`
- Zëvendëson ciklet `for` të thjeshta për ndërtimin e listave.

List Comprehension: Kopjim i listës

Shembull

Krijo një kopje të listës ekzistuese.

```
list2 = [item for item in [1, 2, 3, 4]]
```



Teori

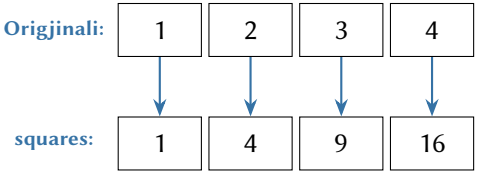
`[item for item in lista]` është ekuivalente me `lista[:]`.

List Comprehension: Transformim (katrorët)

Shembull

Krijo një listë me katrorët e çdo elementi.

```
squares = [x**2 for x in [1, 2, 3, 4]]
print(squares)
```



Teori

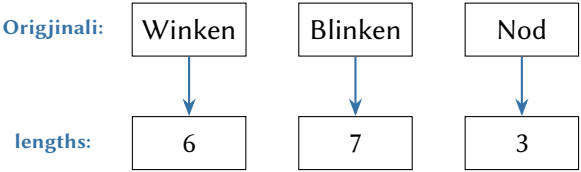
`[x**2 for x in lista]` transformon çdo element sipas shprehjes.

List Comprehension: Transformim (gjatësia e stringjeve)

Shembull

Krijo një listë me gjatësinë e çdo stringu.

```
lengths = [len(s) for s in ['Winken', 'Blinken', 'Nod']]
```



Teori

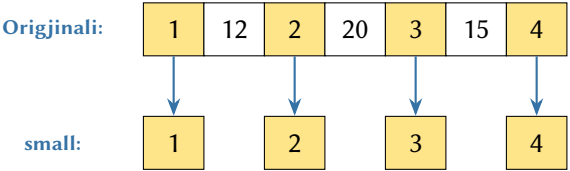
`[len(s) for s in lista_str]` aplikon funksionin `len` për çdo element.

List Comprehension: Filtër (elementë < 10)

Shembull

Krijo një listë me vetëm elementët më të vegjël se 10.

```
small = [n for n in [1, 12, 2, 20, 3, 15, 4] if n < 10]
```



Teori

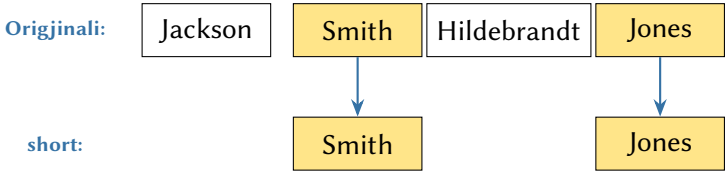
`if n < 10` përfshin vetëm elementët që plotësojnë kushtin.

List Comprehension: Filtër (stringje të shkurtra)

Shembull

Krijo një listë me vetëm stringjet me gjatësi më të vogël se 6.

```
short = [n for n in ['Jackson', 'Smith', 'Hildebrandt', 'Jones'] if len(n) < 6]
```



Teori

Kushti `if len(n) < 6` përzgjedh vetëm stringjet e shkurtra.

Kur të përdorësh list comprehensions?

Këshilla

Përdori list comprehensions kur logjika është e *thjeshtë dhe e lexueshme*. Për raste më të ndërlikuara, përdor cikle të zakonshme `for`.

Përmbajtja

1. Sekuenca dhe Hyrje në Lista
2. Operatori * dhe Iterimi
3. Indeksimi dhe `len()`
4. Mutability, Bashkimi dhe Slicing
5. Kërkimi, Kopjimi dhe Përpunimi
6. Punë me lista dhe skedarë
7. List Comprehensions
8. Metodat e Listave & Funksione të dobishme
9. Listat Dy-Dimensionale
10. Tuplet

Disa metoda të zakonshme

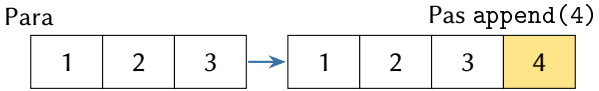
`append(item)`	Shton në fund.
`index(item)`	Indeksi i paraqitjes së parë (`ValueError` nëse s'gjendet).
`insert(i,x)`	Fut në pozicionin `i`, zhvendos djathtas.
`sort()`	Rendit në rritje (*in place*).
`remove(x)`	Heq paraqitjen e parë (`ValueError` nëse s'gjendet).
`reverse()`	Përmbys *in place*.
`min` / `max`	Vlerat skajore.

append — shto në fund

Përshkrim

append është metoda më e përdorur për të shtuar një element të ri në fund të një liste ekzistuese. Kjo metodë ndryshon listën në vend (*in place*) dhe nuk kthen vlerë.

```
nums = [1, 2, 3]
nums.append(4)
```

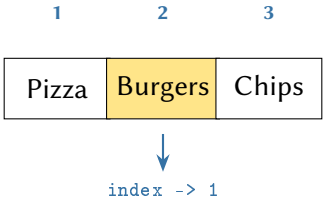


index — gjej pozicionin e një elementi

Përshkrim

Metoda **index** kërkon për një vlerë të caktuar në listë dhe kthen indeksin e paraqitjes së parë të saj. Nëse elementi nuk gjendet, ngrihet përjashtimi `ValueError` .

```
food = ['Pizza', 'Burgers', 'Chips']
pos = food.index('Burgers')
food[pos] = 'Pickles'
```



`index` — Gabime të mundshme

Gabimi `ValueError`

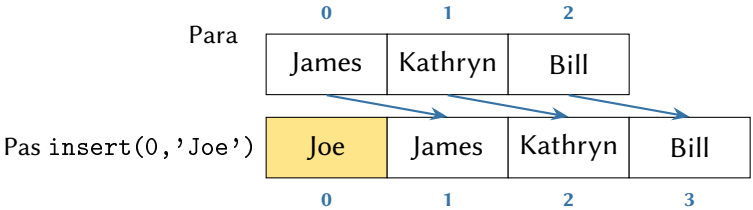
`list.index(x)` hedh `ValueError` nëse `x` nuk gjendet në listë. Përdor `if x in lista: ...` përpara, ose `try/except`.

insert — fut në pozicion dhe zhvendos djathtas

Përshkrim

Metoda **insert** lejon të shtosh një element në një pozicion të caktuar të listës. Elementët që ndodhen në atë pozicion dhe më tej zhvendosen një pozicion djathtas.

```
names = ['James', 'Kathryn', 'Bill']
names.insert(0, 'Joe')
```



`insert` — Indekse jashtë kufijve

Indekse jashtë kufijve

`insert` nuk hedh përjashtim për indekse "të këqija": indeksi $>=$ gjatësisë fut në fund; një indeks negativ shumë i vogël fut në fillim.

sort — rendit në rritje

Metoda **sort** rendit elementët e listës në mënyrë rritëse (nga më i vogli te më i madhi). Kjo metodë ndryshon listën ekzistuese dhe nuk kthen vlerë. Për të marrë një listë të re të renditur, përdor funksionin `sorted(lista)`.

```
a = [9, 1, 0, 2, 8, 6, 7, 4, 5, 3]
a.sort()
```

Para

9	1	0	2	8	6	7	4	5	3
0	1	2	3	4	5	6	7	8	9

Pas `sort()`

`sort` — Renditje *in place*

Renditje *in place*

`list.sort()` ndryshon listën në vend. Për të marrë një kopje të renditur, përdor `sorted(lista)` që kthen listë të re.

remove — hiq paraqitjen e parë

Metoda **remove** heq nga lista paraqitjen e parë të një elementi të caktuar. Nëse elementi nuk gjendet, ngrihet përjashtimi `ValueError`. Elementët që ndodhen pas tij zhvendosen një pozicion majtas.

```
food = ['Pizza', 'Burgers', 'Chips', 'Burgers']
food.remove('Burgers')
```

Para

Pizza	Burgers	Chips	Burgers

Pas `remove('Burgers')`

Pizza	Chips	Burgers

`remove` — Gabime të mundshme

Nëse s'gjendet

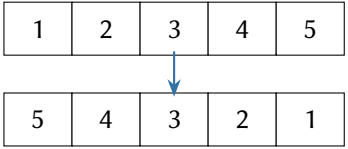
`remove(x)` hedh `ValueError` nëse `x` nuk është në listë.

reverse dhe del

reverse përmbys rendin e listës ekzistuese. **del** fshin një element sipas indeksit.

```
a = [1,2,3,4,5]; a.reverse()
a = [1,2,3,4,5]; del a[2]
```

reverse:



del:

min dhe max

min dhe **max** kthejnë vlerat skajore të listës.

```
vals = [5,4,3,2,50,40,30]
```

5	4	3	2	50	40	30
			min	max		

Përmbajtja

1. Sekuenca dhe Hyrje në Lista
2. Operatori * dhe Iterimi
3. Indeksimi dhe `len()`
4. Mutability, Bashkimi dhe Slicing
5. Kërkimi, Kopjimi dhe Përpunimi
6. Punë me lista dhe skedarë
7. List Comprehensions
8. Metodat e Listave & Funksione të dobishme
9. Listat Dy-Dimensionale
10. Tuplet

Listat 2D — përkufizim & shembull

Ideja

Listë ku çdo element është vetë listë (tabelë rreshta×kolona).

```
students = [['Joe','Kim'], ['Sam','Sue'], ['Kelly','Chris']]
```

Skemë: `students` si tabelë 3 ×2

	Kolona 0	Kolona 1
Rreshti 0	Joe	Kim
Rreshti 1	Sam	Sue
Rreshti 2	Kelly	Chris

Shembull: Matricat e notave

```
scores = [[0,0,0],[0,0,0],[0,0,0]]
```

	Kolona 0	Kolona 1	Kolona 2
Rreshti 0	0	0	0
Rreshti 1	0	0	0
Rreshti 2	0	0	0

Adresimi dhe iterimi *row-major*

```
scores[0][0]; scores[0][1]; scores[0][2]
scores[1][0]; scores[1][1]; scores[1][2]
scores[2][0]; scores[2][1]; scores[2][2]
```

	Kolona 0	Kolona 1	Kolona 2
Rreshti 0	scores[0][0]	scores[0][1]	scores[0][2]
Rreshti 1	scores[1][0]	scores[1][1]	scores[1][2]
Rreshti 2	scores[2][0]	scores[2][1]	scores[2][2]

Programi 13 ((two_dimensional_list.py))

```python
def main():
    # Krijo një listë 3x3.
    values = [[1, 2, 3],
              [10, 20, 30],
              [100, 200, 300]]

    # Shfaq elementët (rresht pas rreshti).
    for row in values:
        for element in row:
            print(element, end=', ')
        print()
```

```
1, 2, 3,
10, 20, 30,
100, 200, 300,
```

Program 13 - Skemë e listës 2D `values`

	Kolona 0	Kolona 1	Kolona 2
Rreshti 0	1	2	3
Rreshti 1	10	20	30
Rreshti 2	100	200	300

Program 14 ((random_number.py))

```python
import random

ROWS = 3
COLS = 4

def main():
    values = [[0 for _ in range(COLS)] for _ in range(ROWS)]

    # Mbush me numra të rastësishëm.
    for r in range(ROWS):
        for c in range(COLS):
            values[r][c] = random.randint(1, 100)

    print(values)
```

List Comprehension e Folezuar për Matrica 2D I

```
values = [[0 for _ in range(COLS)] for _ in range(ROWS)]
```

Çfarë bën ky kod?

Ky është një **list comprehension i folezuar** që krijon një listë 2D (matricë) me zero, me `ROWS` rreshta dhe `COLS` kolona.

- Pjesa e brendshme `[0 for _ in range(COLS)]` krijon **një rresht** me `COLS` zero.
- Pjesa e jashtme `[ ... for _ in range(ROWS)]` përsërit këtë rresht **ROWS** herë, duke ndërtuar matricën.
- Emri `_` është thjesht një **variabël "placeholder"**: tregon që vlera e saj nuk përdoret.

List Comprehension e Folezuar për Matrica 2D II

Shembull: nëse `ROWS = 3` dhe `COLS = 4`:

```
values = [[0, 0, 0, 0],
          [0, 0, 0, 0],
          [0, 0, 0, 0]]
```

Ekuivalente me cikle të zakonshme:

```
values = []
for r in range(ROWS):
    row = []
    for c in range(COLS):
        row.append(0)
    values.append(row)
```

List Comprehension e Folezuar për Matrica 2D III

Këshillë: Edhe kjo formë është korrekte dhe më e shkurtër:

```
values = [[0]*COLS for _ in range(ROWS)]
```

Kujdes: `[[0]*COLS]*ROWS` *duhet shmangur* sepse të gjitha rreshtat do të jenë referenca të së njëjtës listë (ndryshimi i një elementi i ndryshon të gjitha rreshtat).

Program 14 - Skemë e listës 2D `values` - Para ciklit

	Kolona 0	Kolona 1	Kolona 2
Rreshti 0	0	0	0
Rreshti 1	0	0	0
Rreshti 2	0	0	0

Program 14 - Skemë e listës 2D `values` - Pas ciklit

	Kolona 0	Kolona 1	Kolona 2
Rreshti 0	1	5	12
Rreshti 1	76	76	99
Rreshti 2	14	33	6

Përmbajtja

1. Sekuenca dhe Hyrje në Lista
2. Operatori * dhe Iterimi
3. Indeksimi dhe `len()`
4. Mutability, Bashkimi dhe Slicing
5. Kërkimi, Kopjimi dhe Përpunimi
6. Punë me lista dhe skedarë
7. List Comprehensions
8. Metodat e Listave & Funksione të dobishme
9. Listat Dy-Dimensionale
10. Tuplet

Tuplet — përkufizim dhe karakteristika

Përkufizim

Tuple është një *sekuencë* shumë e ngjashme me listën, por **immutable** (e pandryshueshme): pasi krijohet, përmbajtja *nuk* mund të ndryshohet.

Një tuple është një koleksion i renditur elementësh, të cilët mund të jenë të tipeve të ndryshme (numra, stringje, etj.), por ndryshe nga lista, nuk mund të ndryshohet pas krijimit. Kjo do të thotë se nuk mund të shtosh, heqësh ose ndryshosh elementë të tuplet. Tuplet janë të dobishme kur dëshiron të ruash të dhëna që nuk duhet të ndryshohen gjatë ekzekutimit të programit.

Krijimi dhe shfaqja e tupleve

Sintaksa

Tuplet krijohen me kllapa të rrumbullakëta `( ... )`, por Python i lejon edhe pa kllapa në shumë raste, nëse elementët ndahen me presje.

```
my_tuple = (1, 2, 3, 4, 5)
print(my_tuple)

another_tuple = 10, 20, 30
print(another_tuple)
```

Output:

```
(1, 2, 3, 4, 5)
(10, 20, 30)
```

Iterimi me `for`

Ashtu si listat, tuplet mund të iterohen direkt.

```
names = ('Holly', 'Warren', 'Ashley')
for n in names:
    print(n)
```

Output:

```
Holly
Warren
Ashley
```

Indeksimi dhe `range(len(...))`

Tuplet mbështesin indeksimin (nga `0` si zakonisht).

```
names = ('Holly', 'Warren', 'Ashley')
for i in range(len(names)):
    print(names[i])
```

Output:

```
Holly
Warren
Ashley
```

Çfarë mbështesin tuplet

- Tuplet mbështesin pothuajse të gjitha operacionet e listave që *nuk* ndryshojnë përmbajtjen: mund të lexosh elementë me indeks, të marrësh një pjesë të tuplet me slicing, të përdorësh operatorët `in` / `not in`, të bashkosh tuplet me `+` ose të përsëritësh me `*`.
- Mund të përdorësh funksionet `len`, `min`, `max`, dhe metodën `index` për të gjetur pozicionin e një vlere.

Çfarë *nuk* mbështesin tuplet

Ndryshe nga listat, tuplet nuk kanë metoda që ndryshojnë përmbajtjen, si `append`, `remove`, `insert`, `reverse`, `sort`. Çdo përpjekje për caktim me indeks shkakton `TypeError`.

```
t = (1, 2, 3)
t[0] = 99
```

Output:

```
TypeError: 'tuple' object does not support item assignment
```

Tuple me një element

Për të krijuar një tuple me një element të vetëm, duhet të përdorësh një presje pas vlerës, përndryshe Python e konsideron si vlerë të zakonshme, jo tuple.

```
one_el = (1,) # tuple me një element
not_tuple = (1) # thjesht numri 1 (jo tuple)
```

Pse të përdorim tuplet?

Tuplet janë të dobishme në situata kur dëshiron të ruash të dhëna që nuk duhet të ndryshohen, si koordinata (x, y), data, ose rezultate që nuk duhet të modifikohen aksidentalisht. Përveç kësaj, tuplet janë më të shpejta se listat për përpunim, sepse janë struktura më të thjeshta dhe të pandryshueshme. Në shumë raste, Python i përdor tuplet për të kthyer disa vlera nga një funksion, ose për të kaluar të dhëna të pandryshueshme si argument.

Konvertimi mes listave dhe tupleve

Përdor `list()` për të kthyer një tuple në listë, dhe `tuple()` për të kthyer një listë në tuple.

```
number_tuple = (1, 2, 3)
number_list = list(number_tuple)
print(number_list)

str_list = ['one', 'two', 'three']
str_tuple = tuple(str_list)
print(str_tuple)
```

Output:

```
[1, 2, 3]
('one', 'two', 'three')
```

Shembuj përdorimi të tupleve

Tuplet përdoren shpesh për të ruajtur të dhëna të lidhura që nuk duhet të ndryshohen, si p.sh. koordinata gjeografike, data, ose rezultate të pandryshueshme nga funksionet. Për shembull:

```
point = (4, 5)
date = (2024, 6, 1)
```

Kthimi i shumë vlerave nga funksionet

Gjithashtu, funksionet mund të kthejnë disa vlera njëherësh duke përdorur tuple:

```
def min_max(values):
    return (min(values), max(values))

nums = [3, 7, 2, 9]
lo, hi = min_max(nums)
print(lo, hi)
```

Output:

```
2 9
```

Në këtë shembull, funksioni `min_max` kthen një tuple me vlerat minimale dhe maksimale nga lista e dhënë. Kjo është një mënyrë e thjeshtë dhe e qartë për të kthyer shumë vlera nga një funksion. Variabli `lo` merr vlerën minimale dhe `hi` vlerën maksimale.

Përmbledhje

- Listat janë sekuenca të ndryshueshme, tuplet janë të pandryshueshme.
- Mund të përdorësh indekse, slicing, operatorë (`+`, `*`, `in`), dhe funksione si `len()`, `min()`, `max()` për të dyja.
- Listat kanë metoda të shumta për modifikim (`append`, `insert`, `remove`, `sort`, etj.).
- Kopjimi i listave duhet bërë me kujdes (`copy()`, `list()`, `[:]`, etj.).
- Mund të ruash dhe lexosh lista nga skedarë me `write`, `writelines`, `readlines`.
- List comprehensions ofrojnë mënyrë të shkurtër për ndërtimin dhe filtrimin e listave.
- Listat 2D janë lista të listave, të dobishme për tabela dhe matrica.
- Tuplet përdoren kur të dhënat nuk duhet të ndryshohen.

Faleminderit!

Pyetje?